

LibTorch: Day 6

Here are some C++ and mathematical concepts we should know, before proceeding forward with Autograd $\rightarrow$

1) auto keyword

```
auto x = torch::tensor(3.0);
```

 // compiler figures out the type

2) References vs Values

```
torch::Tensor a = torch::ones({3});
```

```
torch::Tensor b = a;
```

 // b is NOT a copy, both point to same data!

```
torch::Tensor c = a.clone();
```

 // c is a real copy. Independent

*Imagine 'a' is a Google Doc. $b=a$ gives the same link - they edit YOUR doc. $a.clone()$ prints it out - now they have their own copy.

Why it matters for autograd?

$\rightarrow$ If you accidentally share tensor data, gradients flow where you don't expect.

3) 'const' and '&' in function arguments

```
void compute(torch::Tensor t) { ... }
```

↳ // BAD - copies the whole tensor
(slow, breaks grad graph)

```
void compute(const torch::Tensor& t) { ... }
```

↳ // GOOD - passes a reference, no
copy

* Always pass tensors by const reference in functions.
Tensors can be huge. Copying them is expensive
and can break the computation graph.

CRUCIAL MATH NEEDED TO UNDERSTAND AUTOGRAD

1) What a derivative actually is?

suppose, if we have $y = w^2$, the derivative $\frac{dy}{dw}$

answers: "If we nudge w by a tiny amount, how much
does y change?"

suppose,

$$y = w^2$$

$$dy/dw = 2w$$

If $w = 3$: $dy/dw = 6$

// Meaning: If we nudge w
by 0.001, y changes by
roughly 0.006

2) The Chain Rule — The Entire foundation of backprop

suppose if we have a chain of operations:

$$x \rightarrow [\text{Operation A}] \rightarrow z \rightarrow [\text{Operation B}] \rightarrow y$$

$$\text{Then } dy/dx = (dy/dz) \times (dz/dx)$$

Example:

$$x = 2$$

$$z = x^3$$

$$\rightarrow \frac{dz}{dx} = 3x^2 = 12$$

$$y = z + 5$$

$$\rightarrow \frac{dy}{dz} = 1$$

$$\frac{dy}{dx} = \frac{dy}{dz} \times \frac{dz}{dx} = 1 \times 12 = 12$$

A neural network is a LONG chain of operations. Backprop applies the chain rule automatically from the output all the way back to every weight.

This is literally what `backward()` does.

LibTorch builds a graph of every operation you do ("the computation graph"). When you call `backward()`, it transverses that graph in reverse applying the chain rule at each step.

3) Partial Derivatives - when there are multiple inputs

Suppose we have code $\rightarrow$

$$y = w \times x^2 + 3$$

There are two inputs: 'w' and 'x'. A partial derivative just means "treat everything else as a constant and differentiate with respect to one thing at a time."

$$\frac{dy}{dw} = x^2 \quad (\text{treat } x \text{ as a constant} \rightarrow x^2 \text{ is just a number})$$

$$\frac{dy}{dx} = 2wx \quad (\text{treat } w \text{ as a constant} \rightarrow \text{power rule on } x^2)$$

With $x=3$, $w=2$:

$$\frac{dy}{dw} = 3^2 = 9 \quad \rightarrow \text{this is what code would print.}$$

$$\frac{dy}{dx} = 2 \times 2 \times 3 = 12 \quad \rightarrow \text{LibTorch computes that too if } x \text{ tracks grad}$$

So, this is all the math concepts we needed for Autograd $\rightarrow$ derivative, chain rule, partial derivative.